

Name: Abhinav Garg
ID: 2023UCP1706

Ans.1.

Code:

```
#include <stdio.h>
#include <omp.h>

void subprogram()
{
    printf("Subprogram outside parallel region, thread %d\n", omp_get_thread_num());
    #pragma omp parallel
    {
        printf("Subprogram inside parallel region, thread %d\n", omp_get_thread_num());
    }
}

int main()
{
    printf("Main outside parallel region thread %d\n", omp_get_thread_num());
    subprogram();
    #pragma omp parallel
    {
        printf("Main inside parallel region thread %d\n", omp_get_thread_num());
        subprogram();
    }
    return 0;
}
```

Output:

With OMP_NESTED=true,

```
mnit@mnit-OptiPlex-5040:~/Desktop/pdc3$ gcc -o ans1 -fopenmp ans1.c
mnit@mnit-OptiPlex-5040:~/Desktop/pdc3$ ./ans1
Main outside parallel region thread 0
Subprogram outside parallel region, thread 0
Subprogram inside parallel region, thread 0
Subprogram inside parallel region, thread 1
Main inside parallel region thread 0
Subprogram outside parallel region, thread 0
Main inside parallel region thread 1
Subprogram outside parallel region, thread 1
Subprogram inside parallel region, thread 0
Subprogram inside parallel region, thread 1
Subprogram inside parallel region, thread 0
Subprogram inside parallel region, thread 1
```

With OMP_NESTED=false,

```
mnit@mnit-OptiPlex-5040:~/Desktop/pdc3$ export OMP_NESTED=false
mnit@mnit-OptiPlex-5040:~/Desktop/pdc3$ gcc -o ans1 -fopenmp ans1.c
mnit@mnit-OptiPlex-5040:~/Desktop/pdc3$ ./ans1
Main outside parallel region thread 0
Subprogram outside parallel region, thread 0
Subprogram inside parallel region, thread 0
Subprogram inside parallel region, thread 1
Main inside parallel region thread 0
Subprogram outside parallel region, thread 0
Main inside parallel region thread 1
Subprogram outside parallel region, thread 1
Subprogram inside parallel region, thread 0
Subprogram inside parallel region, thread 0
```

Ans.2.

Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <omp.h>
```

```
#define N 1000
```

```
int main()
{
    int i;
    int v1[N], v2[N];
    int dp = 0;
    int ds = 0;
    srand(1);
    for (i = 0; i < N; i++) {
        v1[i] = rand() % 100 + 1;
        v2[i] = rand() % 100 + 1;
    }

    omp_set_nested(1);
    #pragma omp parallel reduction(+:dp)
    {
        int id = omp_get_thread_num();
        int n = omp_get_num_threads();
        int chunk_size = N / n;
        int start = id * chunk_size;
        int end = (id == n - 1) ? N : start + chunk_size;
        int local_sum = 0;

        #pragma omp parallel for reduction(+:local_sum)
        for (int j = start; j < end; j++) {
            local_sum += v1[j] * v2[j];
        }
    }
}
```

```
    }  
    dp += local_sum;  
}  
  
for (i = 0; i < N; i++) {  
    ds += v1[i] * v2[i];  
}  
  
printf("Parallel : %d\n", dp);  
printf("Serial : %d\n", ds);  
  
return 0;  
}
```

Output:

```
mnit@mnit-OptiPlex-5040:~/Desktop/pdc3$ gcc -o ans2 -fopenmp ans2.c  
mnit@mnit-OptiPlex-5040:~/Desktop/pdc3$ ./ans2  
Parallel : 2598904  
Serial : 2598904
```